

Guía de instalación y puesta en marcha de CashTrackr

Desde el cambio de versión de PHP hasta crear y ejecutar correctamente el proyecto Laravel

Componente	Configuración final
PHP	8.5.10 — x64 ZTS, Visual C++ 2022
PHP activo	C:\php-8.5.10\php.exe
PHP anterior	C:\php\php.exe — 8.2.20
Laravel	12.69.2
Composer	2.10.3
Base de datos	PostgreSQL en Neon
Proyecto	C:\Users\PC\Desktop\cashtrackr
Laravel	http://127.0.0.1:8000
Vite	http://localhost:5173/

1. Objetivo

Esta guía documenta el procedimiento utilizado para preparar el entorno de desarrollo de CashTrackr en Windows, utilizando PHP 8.5.10, Composer 2.10.3, Laravel 12.69.2 y PostgreSQL en Neon. Incluye los problemas encontrados durante la instalación y las soluciones aplicadas.

2. Comprobar la versión actual de PHP

Primero se verificó qué versión y qué ejecutable estaba tomando Windows.

```
php -v
where php
```

El entorno anterior tenía PHP 8.2.20 en C:\php\php.exe. Para CashTrackr se decidió utilizar PHP 8.5.10 en C:\php-8.5.10.

3. Cambiar el PHP utilizado por la sesión actual de CMD

Para probar PHP 8.5.10 sin modificar permanentemente el PATH de Windows:

```
set "PATH=C:\php-8.5.10;%PATH%"
```

Después se comprobó:

```
php -v
where php
```

El comando set modifica el PATH solamente para la ventana actual de CMD. Al cerrar esa ventana, la configuración anterior vuelve a estar disponible.

4. Configurar las extensiones de PHP

El archivo utilizado fue C:\php-8.5.10\php.ini. Se habilitaron y verificaron:

- openssl
- curl
- fileinfo
- mbstring
- pdo_pgsql
- pgsql

Para comprobar las extensiones activas:

```
php -m
```

Cuando una extensión estaba deshabilitada, se quitó el punto y coma (;) de su línea en php.ini, se guardó el archivo y se volvió a ejecutar php -m.

5. Comprobar y actualizar Composer

Se comprobó Composer con:

```
composer --version
```

La versión inicial era 2.7.4. Debido a los mensajes de deprecación al utilizar PHP 8.5.10, se actualizó Composer.

```
composer self-update
```

En Windows apareció inicialmente “Access is denied” al escribir en C:\ProgramData\ComposerSetup\bin. Se aceptó la solicitud de privilegios de administrador y la operación terminó correctamente.

```
composer --version
```

Resultado final: Composer 2.10.3.

No confundir: `composer self-update` actualiza Composer. `composer update` actualiza las dependencias del proyecto y puede modificar `composer.lock`. Para este paso no se utilizó `composer update`.

6. Crear el proyecto Laravel

Desde el Escritorio se ejecutó:

```
cd C:\Users\PC\Desktop  
laravel new cashtrackr
```

En el asistente se seleccionó:

Configuración	Elección
Starter kit	None / No starter kit
Frontend stack	Blade

7. Errores de extensiones durante la instalación

Durante la creación del proyecto, el instalador solicitó extensiones que todavía no estaban habilitadas. La solución fue habilitarlas en `php.ini` y volver a comprobarlas.

Problema	Solución
openssl faltante	Habilitar <code>extension=openssl</code> y comprobar <code>php -m</code> .
curl faltante	Habilitar <code>extension=curl</code> y comprobar <code>php -m</code> .
fileinfo faltante	Habilitar <code>extension=fileinfo</code> y comprobar <code>php -m</code> .

8. Verificar el proyecto

```
cd C:\Users\PC\Desktop\cashtrackr  
php artisan --version  
php artisan about
```

El resultado final mostró Laravel 12.69.2, PHP 8.5.10, Composer 2.10.3 y la conexión PostgreSQL configurada.

9. Configurar PostgreSQL / Neon

CashTrackr se configuró para PostgreSQL en Neon. Se utilizó el endpoint directo de Neon, sin “-pooler”, y SSL requerido. Las credenciales reales deben mantenerse privadas.

La estructura utilizada en .env es:

```
DB_CONNECTION=pgsql
DB_URL=postgresql://USUARIO:CONTRASENA@HOST.neon.tech/neondb?sslmode=require&channel_binding=require
```

La conexión se comprobó desde Laravel y devolvió correctamente la base neondb y el usuario configurado.

10. Ejecutar las migraciones

```
php artisan migrate
php artisan migrate:status
```

Las migraciones iniciales quedaron como Ran: users, cache y jobs. No es necesario repetirlas salvo que se creen nuevas migraciones.

11. Problema con Laravel Pail en Windows

El script original de desarrollo incluía Laravel Pail. Pail requiere la extensión pcntl, que no está disponible en la instalación de PHP utilizada en Windows.

Para que el entorno pudiera iniciar, se dejó el paquete instalado pero se quitó Pail del script dev. La configuración final utiliza Laravel, Queue y Vite.

```
"dev": [
    "Composer\\Config::disableProcessTimeout",
    "npx concurrently -c \"#93c5fd,#c4b5fd,#fb7185\" \"php artisan serve\" \"php artisan queue:listen --tries=1 --timeout=0\" \"npx vite\""
]
```

No fue necesario eliminar laravel/pail; solamente se retiró del comando composer run dev para evitar que la falta de pcntl detuviera los demás procesos.

12. Ejecutar CashTrackr

```
cd C:\Users\PC\Desktop\cashtrackr
composer run dev
```

La ejecución final funcionó correctamente: Laravel inició en http://127.0.0.1:8000, Queue comenzó a procesar trabajos y Vite quedó disponible en http://localhost:5173/.

13. Verificación final

Comprobación	Resultado
php -v	PHP 8.5.10
where php	C:\php-8.5.10\php.exe aparece primero
composer --version	Composer 2.10.3
php artisan --version	Laravel 12.69.2
php artisan migrate:status	Migraciones iniciales: Ran
composer run dev	server + queue + vite funcionando
Navegador	http://localhost:8000

14. Comandos de referencia rápida

```
# Activar PHP 8.5.10 en la sesión actual
set "PATH=C:\php-8.5.10;%PATH%"
```

```
# Verificar PHP
```

```
php -v
where php
php -m
```

```
# Verificar Composer
composer --version
```

```
# Crear proyecto
cd C:\Users\PC\Desktop
laravel new cashtrackr
```

```
# Entrar al proyecto
cd C:\Users\PC\Desktop\cashtrackr
```

```
# Información de Laravel
php artisan about
```

```
# Migraciones
php artisan migrate
php artisan migrate:status
```

```
# Levantar entorno
composer run dev
```

15. Flujo recomendado para futuras instalaciones

- 1. Comprobar PHP con `php -v` y localizar ejecutables con `where php`.
- 2. Activar la versión de PHP que utilizará el proyecto.
- 3. Verificar extensiones con `php -m`.
- 4. Comprobar Composer y usar `composer self-update` si se requiere actualizarlo.
- 5. Crear el proyecto con `laravel new nombre`.
- 6. Entrar al proyecto y ejecutar `php artisan about`.
- 7. Configurar `.env` y comprobar la base de datos.
- 8. Ejecutar migraciones.
- 9. Revisar `composer.json` si algún proceso del entorno no es compatible con Windows.
- 10. Ejecutar `composer run dev` y comprobar el navegador.

Estado final: CashTrackr quedó creado y ejecutándose correctamente con Laravel 12.69.2, PHP 8.5.10, Composer 2.10.3, Vite 7.3.6 y PostgreSQL en Neon.